Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Факультет электронно-информационных систем
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №5

Специальность АС-68

Выполнил:

Студент 3-го курса

Группы АС-68

Дворак В. А

Проверил:

А. А. Крощенко
Ст. преп. кафедры ИИТ,

«___»_____ 2026 г.

Брест 2026

Цель работы: выполнить моделирование прогнозирующей нелинейной ИНС.

Вариант 4

№ варианта	a	b	c	d	Кол-во входов ИНС	Кол-во НЭ в скрытом слое
4	0.4	0.4	0.08	0.4	6	2

Для генерации обучающих и тестовых данных использовать функцию:

$$y = a * \cos(bx) + c * \sin(dx)$$

Код программы:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

a, b, c, d = 0.4, 0.4, 0.08, 0.4
L = 6  # Кол-во входов ИНС
H = 2  # Кол-во НЭ в скрытом слое

epochs = 2000
lr = 0.05
test_frac = 0.25
N = 600

# ГЕНЕРАЦИЯ ВРЕМЕННОГО РЯДА И РАЗДЕЛЕНИЕ
x = np.linspace(0, 20 * np.pi, N)
y = a * np.cos(b * x) + c * np.sin(d * x)

# Разделение выборки
split_idx = int(N * (1 - test_frac))
y_train_raw = y[:split_idx]
y_test_raw = y[split_idx:]
x_train_raw = x[:split_idx]
x_test_raw = x[split_idx:]

# Вычисление параметров нормализации строго по y_train (без утечки данных)
y_mean = y_train_raw.mean()
y_std = y_train_raw.std()

y_train_norm = (y_train_raw - y_mean) / y_std
y_test_norm = (y_test_raw - y_mean) / y_std

# Функция формирования датасета скользящим окном L
def make_dataset(y_series, L):
    X, Y = [], []
    for i in range(L, len(y_series)):
        X.append(y_series[i - L:i])
        Y.append(y_series[i])
    return np.array(X), np.array(Y).reshape(-1, 1)

X_train, Y_train_norm = make_dataset(y_train_norm, L)
X_test, Y_test_norm = make_dataset(y_test_norm, L)

# Эталонные данные для графиков и таблиц в исходном масштабе
Y_train_true = Y_train_norm * y_std + y_mean
Y_test_true = Y_test_norm * y_std + y_mean

x_train_targets = x_train_raw[L:]
x_test_targets = x_test_raw[L:]

# АРХИТЕКТУРА И ФУНКЦИИ АКТИВАЦИИ
def sigmoid(z):
```

```
    z_clip = np.clip(z, -50, 50)
    return 1.0 / (1.0 + np.exp(-z_clip))

def sigmoid_deriv(a_sigmoid):
    return a_sigmoid * (1.0 - a_sigmoid)

# Инициализация весов
rng = np.random.default_rng(42)
W1 = rng.normal(0.0, np.sqrt(1.0 / L), size=(L, H))
b1 = np.zeros((1, H))
W2 = rng.normal(0.0, np.sqrt(1.0 / H), size=(H, 1))
b2 = np.zeros((1, 1))

# Прямой проход
def forward(X, W1, b1, W2, b2):
    z1 = X @ W1 + b1
    a1 = sigmoid(z1)
    z2 = a1 @ W2 + b2
    return z2, (X, z1, a1, z2)

# Обратный проход (все аргументы строго согласованы)
def backward(W1, b1, W2, b2, cache, y_pred, Y, lr):
    X, z1, a1, z2 = cache
    num_samples = X.shape[0]

    err = y_pred - Y
    grad_out = (2.0 / num_samples) * err

    dW2 = a1.T @ grad_out
    db2 = np.sum(grad_out, axis=0, keepdims=True)

    da1 = grad_out @ W2.T
    dz1 = da1 * sigmoid_deriv(a1)

    dW1 = X.T @ dz1
    db1 = np.sum(dz1, axis=0, keepdims=True)

    W1 -= lr * dW1
    b1 -= lr * db1
    W2 -= lr * dW2
    b2 -= lr * db2

    loss = np.mean(err ** 2)
    return W1, b1, W2, b2, loss

# 4. ОБУЧЕНИЕ ИНС
loss_history = []
for epoch in range(1, epochs + 1):
    y_pred, cache = forward(X_train, W1, b1, W2, b2)
    W1, b1, W2, b2, loss = backward(W1, b1, W2, b2, cache, y_pred,
Y_train_norm, lr)
    loss_history.append(loss)
    if epoch % 500 == 0 or epoch == 1:
        print(f"Epoch {epoch:4d} | MSE={loss:.6f}")

#  ПРОГНОЗИРОВАНИЕ И ДЕНОРМАЛИЗАЦИЯ
def predict(X, W1, b1, W2, b2):
    z1 = X @ W1 + b1
    a1 = sigmoid(z1)
    return a1 @ W2 + b2

train_pred_norm = predict(X_train, W1, b1, W2, b2)
test_pred_norm = predict(X_test, W1, b1, W2, b2)
```

```
# Перевод предсказаний в исходный масштаб
train_pred = train_pred_norm * y_std + y_mean
test_pred = test_pred_norm * y_std + y_mean

#  6. ВЫВОД ТАБЛИЦ И ГРАФИКОВ
train_df = pd.DataFrame({
    "Эталонное значение": Y_train_true.flatten(),
    "Полученное значение": train_pred.flatten(),
    "Отклонение": (train_pred - Y_train_true).flatten()
})

test_df = pd.DataFrame({
    "Эталонное значение": Y_test_true.flatten(),
    "Полученное значение": test_pred.flatten(),
    "Отклонение": (test_pred - Y_test_true).flatten()
})

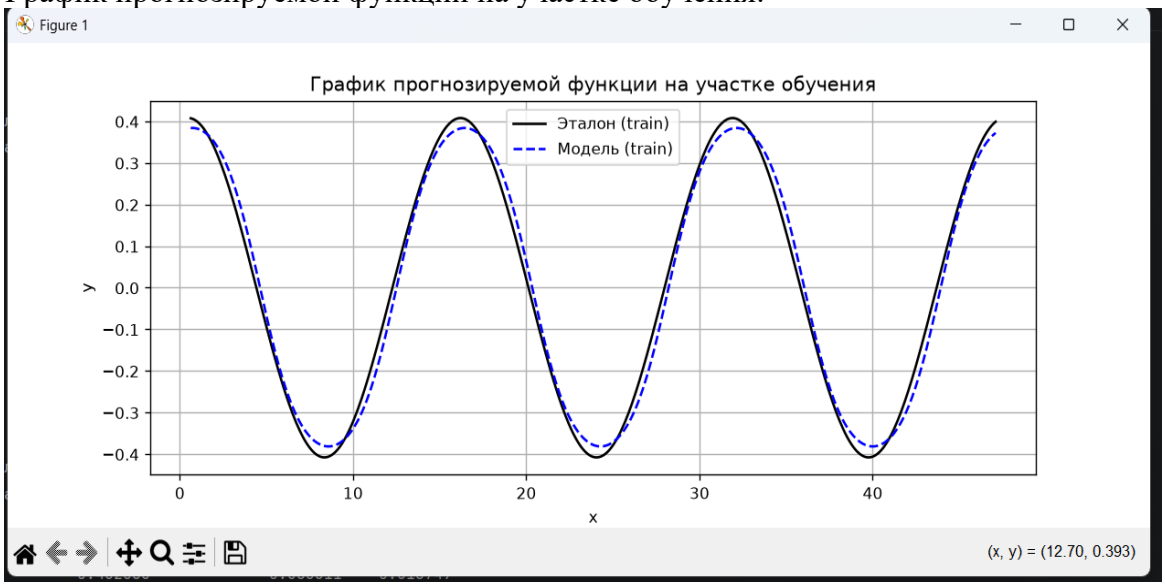
print("\n Результаты обучения ")
print(train_df.head(10))

print("\n Результаты прогнозирования  ")
print(test_df.head(10))

# График 1: Функция на участке обучения
plt.figure(figsize=(10, 4))
plt.plot(x_train_targets, Y_train_true, label="Эталон (train)",
color="black")
plt.plot(x_train_targets, train_pred, label="Модель (train)", color="blue",
linestyle="--")
plt.title("График прогнозируемой функции на участке обучения ")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True)
plt.show()

# График 2: Ошибка MSE по эпохам
plt.figure(figsize=(8, 4))
plt.plot(loss_history, color="red")
plt.title("График изменения ошибки в зависимости от итерации (MSE)")
plt.xlabel("Итерация (Эпоха)")
plt.ylabel("MSE")
plt.grid(True)
plt.show()
```

График прогнозируемой функции на участке обучения:



Изменение ошибки по эпохам:



Результат работы программы:

```
Epoch    1 | MSE=0.359648
Epoch  500 | MSE=0.001522
Epoch 1000 | MSE=0.001294
Epoch 1500 | MSE=0.001084
Epoch 2000 | MSE=0.000896

Таблица обучения (первые 10 строк):
   Эталонное значение  Полученное значение  Отклонение
0            0.407319             0.383830   -0.023489
1            0.406031             0.384037   -0.021994
2            0.404028             0.383792   -0.020236
3            0.401315             0.383094   -0.018221
4            0.397894             0.381938   -0.015957
5            0.393774             0.380320   -0.013454
6            0.388960             0.378233   -0.010727
7            0.383462             0.375669   -0.007793
8            0.377288             0.372618   -0.004671
9            0.370451             0.369067   -0.001383
Таблица прогноза (первые 10 строк):
   Эталонное значение  Полученное значение  Отклонение
0            0.406420             0.384028   -0.022392
1            0.404596             0.383896   -0.020700
2            0.402060             0.383311   -0.018749
3            0.398815             0.382270   -0.016546
4            0.394869             0.380768   -0.014101
5            0.390228             0.378799   -0.011429
6            0.384900             0.376355   -0.008545
7            0.378894             0.373427   -0.005467
8            0.372222             0.370002   -0.002219
9            0.364894             0.366070    0.001176
```

Вывод: выполнил моделирование прогнозирующей нелинейной ИНС.